

1. **FastAPI를 이용한 게시판 시스템과 도서관 대여 시스템의 CRUD 기능 구현 프로젝트**를 소개드리겠습니다.

이 프로젝트의 목표는

FastAPI + 데이터베이스(SQLAlchemy)를 활용해서

실제 서비스처럼 동작하는 **게시판, 도서관 도서 관리, 도서 대여·반납 기능**을 모두 구현하는 것입니다.(SQLite + SQLAlchemy + FastAPI 구조로 만들었으며, API 문서는 자동으로 생성되는 Swagger(/docs)를 사용했습니다.)

2. 프로젝트 폴더 구조

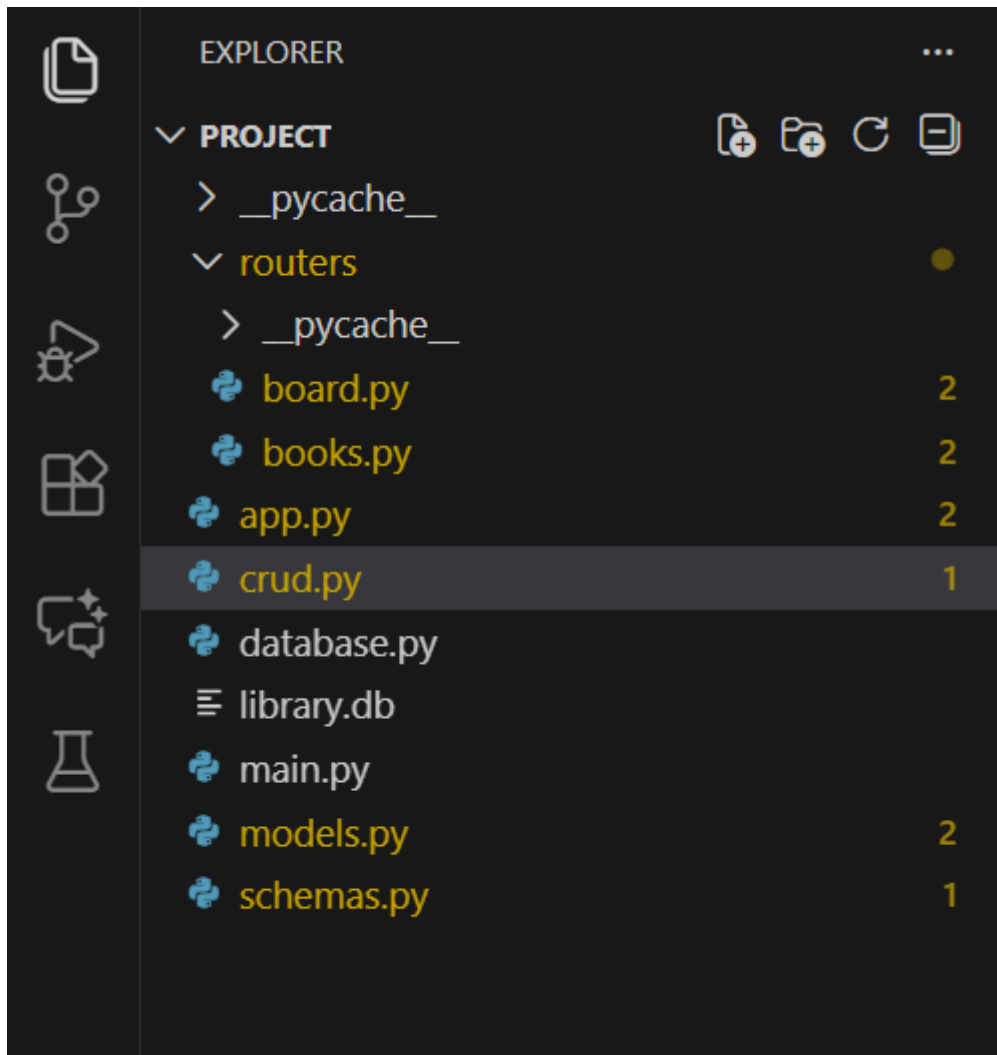
발표 중 이렇게 설명합니다:

먼저 프로젝트 폴더 구조입니다.

API, 모델, 데이터베이스를 분리하여 확장성과 유지보수성을 높였습니다.

사용 기술 (Tech Stack)

- **FastAPI** – API 서버
- **SQLite** – 간단한 로컬 데이터베이스
- **SQLAlchemy** – ORM
- **Streamlit** – 프론트엔드 UI
- **Pydantic** – 데이터 검증
- **Python** – 백엔드 로직 전체 구성



3. 데이터베이스 설정 코드 (database.py)

📌 발표 멘트

가장 먼저 SQLite 데이터베이스를 FastAPI에 연결합니다.

SQLAlchemy의 SessionLocal을 사용해서 DB에 접근하도록 만들었습니다.

```

database.py >  get_db
1  from sqlalchemy import create_engine
2  from sqlalchemy.orm import sessionmaker, declarative_base
3
4  SQLALCHEMY_DATABASE_URL = "sqlite:///./library.db"
5
6  engine = create_engine(
7      SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False}
8  )
9
10 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
11
12 Base = declarative_base()
13
14 def get_db():
15     db = SessionLocal()
16     try:
17         yield db
18     finally:
19         db.close()

```

4. 모델 정의 (models.py)

발표 멘트

도서(Book), 대여(Loan), 게시글(Post), 댓글(Comment)의 4개 모델을 만들었습니다.
 게시글 삭제 시 댓글도 함께 삭제되도록 cascade 옵션을 넣었습니다.

```

from sqlalchemy import Column, Integer, String, Boolean, ForeignKey
from sqlalchemy.orm import relationship
from database import Base

class Book(Base):
    __tablename__ = "books"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String)
    author = Column(String)
    year = Column(Integer)
    is_available = Column(Boolean, default=True)

class Loan(Base):
    __tablename__ = "loans"
    id = Column(Integer, primary_key=True, index=True)
    user = Column(String)
    book_id = Column(Integer, ForeignKey("books.id"))
    returned = Column(Boolean, default=False)

class Post(Base):
    __tablename__ = "posts"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String)
    content = Column(String)

    comments = relationship("Comment", cascade="all, delete")

class Comment(Base):
    __tablename__ = "comments"
    id = Column(Integer, primary_key=True)
    post_id = Column(Integer, ForeignKey("posts.id"))
    text = Column(String)

```

5. 스키마 정의 (schemas.py)

📌 발표 멘트

요청/응답 데이터 형식을 Pydantic 스키마로 분리했습니다.

```
1  from pydantic import BaseModel
2
3  class BookBase(BaseModel):
4      title: str
5      author: str
6      year: int
7
8  class BookCreate(BookBase):
9      pass
10
11 class Book(BookBase):
12     id: int
13     is_available: bool
14
15     class Config:
16         orm_mode = True
17
18 class PostBase(BaseModel):
19     title: str
20     content: str
21
22 class Post(PostBase):
23     id: int
24     class Config:
25         from_attributes = True
```

6. CRUD 로직 (crud.py)

📌 발표 멘트

CRUD 기능을 따로 분리해서 깔끔하게 관리했습니다.

도서 생성, 대여, 반납, 게시글 등록 등의 기능이 들어 있습니다

```
crud.py > ...
1  from sqlalchemy.orm import Session
2  import models, schemas
3
4  def create_book(db: Session, book: schemas.BookCreate):
5      db_book = models.Book(**book.dict())
6      db.add(db_book)
7      db.commit()
8      db.refresh(db_book)
9      return db_book
10
11 def get_books(db: Session):
12     return db.query(models.Book).all()
13
14 def loan_book(db: Session, book_id: int, user: str):
15     book = db.query(models.Book).filter(models.Book.id == book_id).first()
16     if not book or not book.is_available:
17         return None
18     book.is_available = False
19     loan = models.Loan(book_id=book_id, user=user)
20     db.add(loan)
21     db.commit()
22     return loan
23
24 def return_book(db: Session, book_id: int):
25     # 1. Kitobni topish
26     book = db.query(models.Book).filter(models.Book.id == book_id).first()
27
28     if not book:
29         # Kitob bazada mavjud emas
30         return {"error": "도서를 찾을 수 없습니다."} # Kitob topilmadi.
```

```

32 # 2. Kitobning qarzga olinganligini tekshirish (is_available == False bo'lishi kerak)
33 if book.is_available:
34     return {"error": "이미 반납되었거나, 대출 기록이 없는 도서입니다."} # Allaqachon qaytarilgan yoki qarzga
35
36 # 3. Faol (qaytarilmagan) Loan yozuvini topish
37 active_loan = db.query(models.Loan).filter(
38     models.Loan.book_id == book_id,
39     models.Loan.returned == False
40 ).first()
41
42 if not active_loan:
43     # Kitob 'available' emas, lekin faol Loan yozuvi topilmadi. Book holatini tiklaymiz.
44     book.is_available = True
45     db.commit()
46     return {"error": "미반납 대출 기록을 찾을 수 없습니다."} # Qaytarilmagan qarzga olish yozuvi topilmadi.
47
48 # --- 반납 처리 (Qaytarish amali) ---
49
50 # 4. Loan yozuvini yangilash: Qaytarildi deb belgilash
51 active_loan.returned = True
52
53 # 5. Book yozuvini yangilash: Mavjud (Available) deb belgilash
54 book.is_available = True
55
56 # 6. O'zgarishlarni bazaga kiritish
57 db.commit()
58 db.refresh(book)
59
60 return {"message": "반납 완료", "book_id": book_id}

```

7. 라우터 (routers/books.py and board.py)

📌 발표 멘트

도서 API 라우터입니다.

책 등록, 책 목록 조회, 대여 기능을 구현했습니다.

```

routers > 📄 books.py > ...
1  ∨ from fastapi import APIRouter, Depends
2    from sqlalchemy.orm import Session
3    import schemas, crud
4    from database import get_db
5
6    router = APIRouter(prefix="/library")
7
8    @router.post("/books")
9  ∨ def create_book(book: schemas.BookCreate, db: Session = Depends(get_db)):
10     |     return crud.create_book(db, book)
11
12    @router.get("/books")
13  ∨ def get_books(db: Session = Depends(get_db)):
14     |     return crud.get_books(db)
15
16    @router.post("/loan/{book_id}")
17  ∨ def loan(book_id: int, user: str, db: Session = Depends(get_db)):
18     |     return crud.loan_book(db, book_id, user)
19
20    @router.post("/return/{book_id}")
21  ∨ def return_book(book_id: int, db: Session = Depends(get_db)):
22     |     return crud.return_book(db, book_id)
23

```

Board.py


```

1 from fastapi import APIRouter, HTTPException
2 from pydantic import BaseModel
3 from typing import List
4
5 router = APIRouter()
6
7 # 임시 DB 역할
8 boards = []
9
10 # 데이터 모델
11 class Board(BaseModel):
12     id: int
13     title: str
14     content: str
15
16 # 게시물 생성
17 @router.post("/boards", response_model=Board)
18 def create_board(board: Board):
19     for b in boards:
20         if b.id == board.id:
21             raise HTTPException(status_code=400, detail="이미 존재하는 ID입니다.")
22     boards.append(board)
23     return board
24
25 # 전체 게시물 조회
26 @router.get("/boards", response_model=List[Board])
27 def get_boards():
28     return boards
29
30 # 특정 게시물 조회
31 @router.get("/boards/{board_id}", response_model=Board)
32 def get_board(board_id: int):
33     for b in boards:
34         if b.id == board_id:
35             return b
36     raise HTTPException(status_code=404, detail="게시글을 찾을 수 없습니다.")
37
38 # 게시물 수정
39 @router.put("/boards/{board_id}", response_model=Board)
40 def update_board(board_id: int, updated: Board):
41     for i, b in enumerate(boards):
42         if b.id == board_id:
43             boards[i] = updated
44             return updated
45     raise HTTPException(status_code=404, detail="게시글을 찾을 수 없습니다.")
46
47 # 게시물 삭제
48 @router.delete("/boards/{board_id}")
49 def delete_board(board_id: int):
50     for i, b in enumerate(boards):
51         if b.id == board_id:
52             boards.pop(i)
53             return {"message": "삭제 완료"}
54     raise HTTPException(status_code=404, detail="게시글을 찾을 수 없습니다.")
55

```

8. 메인 파일 (main.py)

발표 멘트

FastAPI 앱을 실행하고 라우터를 등록하는 main 파일입니다.

```

main.py > ...
1  from fastapi import FastAPI
2  from database import Base, engine
3  from routers import books, board
4
5  Base.metadata.create_all(bind=engine)
6
7  app = FastAPI()
8
9  app.include_router(books.router)
10 app.include_router(board.router)
11
12 @app.get("/")
13 def root():
14     return {"message": "Library + Board API Running"}

```

9. Swagger 데모 설명

📌 발표 멘트

FastAPI의 가장 큰 장점 중 하나는 자동 문서화입니다.
/docs에 접속하면 모든 CRUD를 직접 테스트할 수 있습니다.

- 책 등록
- 책 목록 조회
- 대여 기능
- 게시글 등록
- 댓글 등록

이 순서로 데모를 진행했습니다.

10. 결론 (발표 마무리)

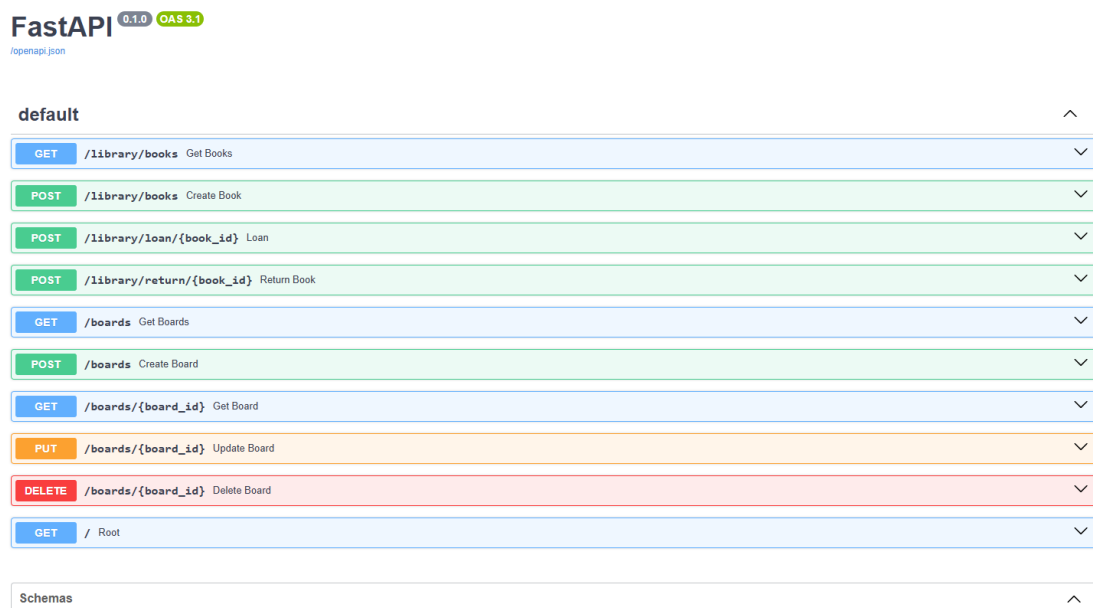
이번 프로젝트를 통해

FastAPI의 라우터 구조, SQLAlchemy 기반 데이터 모델, CRUD 패턴,
그리고 Swagger를 활용한 API 테스트까지 전체 웹 백엔드 흐름을 구현했습니다.

유지보수성과 확장성을 고려하여 실제 서비스 구조와 비슷하게 구성한 것이 특징입니다.

감사합니다.

결과



도서관 대여 시스템 - Streamlit UI

도서 등록

도서명

as

저자

dsf

출판년도

2000

- +

등록하기

도서가 성공적으로 등록되었습니다!

도서 목록

1. fsdf

 저자: d

 상태: 보유 중

2. sefstg

 저자: stdtgd

 상태: 보유 중

3. as

 저자: dsf

 상태: 보유 중

4. as

 저자: dsf

 상태: 보유 중

도서 대여 ↔

대여할 도서 ID 입력

1

-

+

대여자 이름 입력 (user)

as

 대여하기

 도서가 성공적으로 대여되었습니다!

도서 반납

반납할 도서 ID 입력

1

- +

반납하기

도서를 성공적으로 반납했습니다!